TSA Lab Notes

Author: Soham Bhattacharya
Module: Time-Series
Spring 2025

1 01.TScode.R————————————————————————————-

This section summarizes key plots, datasets, and functions used in the exploratory analysis of time series data using the `astsa` package in R. The focus is on visualizing different types of time series, understanding structures like white noise, moving averages, autoregressive processes, and interpreting autocorrelations and cross-correlations.

Important Datasets and Plots

- `jj`: Johnson and Johnson quarterly earnings per share.
- `gtemp_both`: Global land and ocean temperature deviations.
- `speech`: Speech signal sampled at 0.1s interval.
- `nyse`: New York Stock Exchange returns.
- `soi`: Southern Oscillation Index (pressure difference between Tahiti and Darwin).
- `rec`: Recruitment index (e.g., fish entering a population).
- `fmri1`: BOLD signals from fMRI scans of different brain regions.
- `EQ5`, `EXP6`: Earthquake and explosion signals.
- `soiltemp`: 64x36 matrix of surface soil temperatures.

Key R Functions and Concepts

- `plot.ts()`, `ts.plot()`: Plotting time series data.
- `filter()`: Used for moving average and autoregressive processes.
  - `sides=2`: Centered (two-sided) moving average.
  - `method="recursive"`: Used for autoregressive filtering.
- `rnorm(n, mean, sd)`: Generate white noise (Gaussian).
- `cumsum()`: Simulate random walk by accumulating noise.
- `set.seed(seed)`: Fix the randomness for reproducibility.
- `acf()`, `ccf()`: Plot autocorrelation and cross-correlation functions.
- `fft()`: Fast Fourier Transform.

- `persp()`: 3D perspective plot for surfaces like ACF or temperature maps.
- `rowMeans()`: Compute average values for each row in a matrix (e.g., average temperature by row).

Important Figures Explained

- **Figure 1.08**: Demonstrates the effect of smoothing white noise using a moving average filter.
- **Figure 1.09**: Constructs an AR(1) process from white noise.
- **Figure 1.10**: Random walk visualization with drift.
- **Figure 1.11**: Effects of adding white noise to a cosine wave.
- **Figure 1.13, 1.14**: ACF and CCF plots to identify lag structures.
- **Figure 1.15-1.17**: Visual and spectral analysis of spatial data.

Good Practices

- Add padding to time series before filtering to avoid startup issues.
- Use `set.seed()` before simulations to make results reproducible.
- Check dataset structure using `str()`, `summary()`, or `head()`.

2 02.TScode.R————————————————————————————-

1. **Time Series Object Creation**
   - `ts()` function to define a time series
2. **Time Series Plotting**
   - `plot.ts()` to visualize raw and transformed data
3. **Time Series Decomposition Models**
   - Additive Model
   - Multiplicative Model
4. **Log Transformation**
   - Applied to stabilize variance
   - Used to convert multiplicative to additive model
5. **Trend Estimation Techniques**

- Finite (Centered) Moving Average
- Exponential Smoothing
- Polynomial Regression (Linear, Quadratic)

6. Trend Elimination

- De-trending using residuals
- First-order differencing
- Second-order differencing
- Log differencing

7. Seasonality Estimation Techniques

- Harmonic Regression using trigonometric terms:

$$\cos\left(\frac{2\pi kt}{T}\right),\ \sin\left(\frac{2\pi kt}{T}\right)$$

- Seasonal Indices via Moving Average

8. Seasonality Elimination

- Seasonal differencing (e.g., 12-lag difference for monthly data)

9. Combined Trend and Seasonality Elimination

- Moving Average smoothing followed by regression
- Double differencing: Seasonal and first-order

10. R Functions for Regression and Filtering

- `filter()` – Moving Average Filter
- `lm()` – Linear Regression Model
- `abline()` – Add regression line to plot

11. Use of Time and Polynomial Terms in Regression

- Time index as explanatory variable
- Squared time term for quadratic trends

3 03.TScode.R----------

1. **Bernoulli IID Process**

- Binary Process: $b_t \sim \text{Bernoulli}(0.5) \Rightarrow b_t \in \{-1, 1\}$
- Code: b = (2∗rbinom(500,1,0.5)−1)

2. **Simple Random Walk**

- $S_t = \sum_{i=1}^{t} b_i$
- Code: s = cumsum(b)

3. **White Noise Process**
   - $w_t \sim \mathcal{N}(0, 1)$
   - Code: w = rnorm(500,0,1)
4. **Random Walk from White Noise**
   - $r_t = \sum_{i=1}^{t} w_i$
   - Code: r = cumsum(w)
5. **Autocorrelation Function (ACF)**
   - $\rho_k = \frac{\gamma_k}{\gamma_0}$, where $\gamma_k = \text{Cov}(X_t, X_{t-k})$
   - Code: acf(xt, lag.max = 40)
6. **Ljung-Box Test for Randomness**
   - $Q = n(n+2) \sum_{k=1}^{h} \frac{\hat{\rho}_k^2}{n-k}$
   - Code: Box.test(xt, lag = 40, type = "Ljung−Box", fitdf = 0)
7. **Turning Point Test**
   - Tests randomness using: $y_{i-1} < y_i > y_{i+1}$ or $y_{i-1} > y_i < y_{i+1}$
   - Code: turning.point.test(xt)
8. **QQ Plot for Normality**
   - Visual test to compare quantiles of the sample to a normal distribution
   - Code: qqnorm(xt)
9. **Shapiro-Francia Test**
   - Test statistic W', approximates normality
   - Code: sf.test(xt)
10. **Shapiro-Wilk Test**
    - Test statistic $W = \frac{(\sum a_i x_{(i)})^2}{\sum (x_i - \bar{x})^2}$
    - Code: shapiro.test(xt)
11. **Signal + Noise Model**
    - $x_t = \text{signal}(t) + \epsilon_t$, where $\epsilon_t \sim \text{WN}(0, \sigma^2)$
    - Code: xt = t/2 + cos(t/10∗pi/180) + rnorm(200,0,.5)

3.1 Ljung-Box Test

R Function:

```
Box.test(xt, lag = 40, type = "Ljung-Box", fitdf = 0)
```

Purpose: Tests whether the time series is *independent* (i.e., autocorrelations up to a certain lag are all zero).

Null Hypothesis (H_0): The data are **i.i.d.** (independent and identically distributed); i.e., no autocorrelation up to the given lag.

Alternative Hypothesis (H_1): The data are **not i.i.d.**, i.e., at least one autocorrelation up to the given lag is non-zero.

Test Statistic:

$$Q = n(n+2)\sum_{k=1}^{h}\frac{\hat{\rho}_k^2}{n-k}$$

where:

- n is the number of observations,
- $\hat{\rho}_k$ is the sample autocorrelation at lag k,
- h is the number of lags.

Distribution: Under H_0, the test statistic Q asymptotically follows a chi-square distribution:

$$Q \sim \chi^2_{h-p}$$

3.2 Turning Point Test

R Function:

```
turning.point.test(xt)
```

Purpose: Tests for *randomness* (no trend or dependence) in a time series.

Definition of a Turning Point: A time point x_t is considered a turning point if:

$$(x_{t-1} < x_t > x_{t+1}) \quad \text{or} \quad (x_{t-1} > x_t < x_{t+1})$$

Null Hypothesis (H_0): The observations are **i.i.d.** — indicating randomness.

Alternative Hypothesis (H_1): The observations are **not i.i.d.** — indicating serial dependence.

Expected Value and Variance under H_0:

$$\mathbb{E}(T) = \frac{2(n-2)}{3}, \quad \text{Var}(T) = \frac{16n-29}{90}$$

A z-score is calculated to test the hypothesis and is compared against the standard normal distribution.

3.3 QQ Plot for Normality Testing

The Quantile-Quantile (QQ) plot is a graphical tool to assess whether a dataset follows a specified theoretical distribution, commonly the normal distribution. In R, the command used is:

```
qqnorm(xt)
qqline(xt)
```

This plot compares the quantiles of the data with the quantiles of a standard normal distribution. If the data are normally distributed, the points will approximately lie on a straight line.

3.4 Shapiro-Francia Test for Normality

The Shapiro-Francia test is a statistical test to evaluate the hypothesis that a sample comes from a normally distributed population. It is particularly suitable for larger sample sizes. The test statistic is based on the correlation between the ordered sample values and the expected values of order statistics from a standard normal distribution.

```
library(nortest)
sf.test(xt)
```

Null Hypothesis (H_0): The data is normally distributed.
Alternative Hypothesis (H_1): The data is not normally distributed.

The test returns a test statistic and a corresponding p-value. A small p-value (typically less than 0.05) indicates strong evidence against the null hypothesis, suggesting that the data is not normally distributed.

4 04.TScode.R————————————————————————————-

4.1 White Noise Process

A white noise process consists of uncorrelated random variables with mean zero and constant variance. It is used as a fundamental building block in time series models.

```
w = rnorm(500,0,1)
plot.ts(w, main="white noise")
```

4.2 Moving Average Processes

4.2.1 MA(1) Process

A first-order moving average (MA(1)) process is defined as:

$$X_t = W_t + \theta W_{t-1}$$

```
theta = -0.8
ma1 = w[2:500] + theta * w[1:499]
plot.ts(ma1, main="First-order moving average")
```

4.2.2 MA(2) Process

A second-order moving average (MA(2)) process:

$$X_t = W_t + \theta_1 W_{t-1} + \theta_2 W_{t-2}$$

```
theta1 = -0.8
theta2 = 0.6
ma2 = w[3:500] + theta1 * w[2:499] + theta2 * w[1:498]
plot.ts(ma2, main="Second-order moving average")
```

4.3 Autoregressive Processes

4.3.1 AR(1) Process

An autoregressive process of order 1 is:

$$X_t = \phi X_{t-1} + W_t$$

```
phi = 0.9
ar1[i] = phi * ar1[i-1] + w[i]
```

4.4 AR(2) Process

An autoregressive process of order 2 is:

$$X_t = \phi_1 X_{t-1} + \phi_2 X_{t-2} + W_t$$

```
ar2[i] = phi1 * ar2[i-1] + phi2 * ar2[i-2] + w[i]
```

4.5 ARMA(1,1) Process

The ARMA(1,1) process combines both AR and MA terms:

$$X_t = \phi X_{t-1} + W_t + \theta W_{t-1}$$

```
arma_1_1[i] = phi * arma_1_1[i-1] + w[i] + theta * w[i-1]
```

4.6 Autocorrelation Function (ACF)

The ACF measures correlation of a time series with its lagged values. Used to identify MA order.

```
acf(w, main="white noise acf")
acf(ma1, main="MA(1) acf")
acf(ma2, main="MA(2) acf")
acf(ar1, main="AR(1) acf")
acf(ar2, main="AR(2) acf")
acf(arma_1_1, main="ARMA(1,1) acf")
```

4.7 Theoretical Bounds and Confidence Intervals for ACF

Confidence bounds for estimated ACF values help assess significance of autocorrelation.

```
abline(a=1.96*sqrt(w22hat/length(ma1)), b=0, col='red', lty=2)
```

4.8 Fitting Trend in Real Data (Lake Huron)

```
trFit <- lm(LakeHuron - 570 ~ time(LakeHuron))
plot(as.numeric(time(LakeHuron)), trFit$residuals)
```

4.9 ACF of Residuals from Trend Fit

```
acf(trFit$residuals, main="Lake Huron residual acf")
```

4.10 Partial Autocorrelation Function (PACF)

PACF removes indirect effects of lags and is used to identify AR order.

```
pacf(w, main="white noise pacf")
pacf(ma1, main="MA(1) pacf")
pacf(ar1, main="AR(1) pacf")
...
```

4.11 PACF of Real Data (Sunspots)

```
pacf(sunspot.year[70:169])
```

5 05.TScode.R————————————————————————————-

5.1 Time Series: Fitting by `arma()` Method

5.1.1 Autoregressive Process

The autoregressive (AR) process models a time series where the current value is based on the previous value, with a specific autoregressive coefficient ϕ.

```
par(mfrow=c(2,1))
plot(arima.sim(list(order=c(1,0,0), ar=.9), n=100), ylab="x",
main=(expression(AR(1)~~~phi==+.9)))
plot(arima.sim(list(order=c(1,0,0), ar=-.9), n=100), ylab="x",
main=(expression(AR(1)~~~phi==-.9)))
```

5.1.2 Moving Average Process

A moving average (MA) process models a time series based on a linear combination of previous white noise error terms. It is characterized by a moving average coefficient θ.

```
par(mfrow = c(2,1))
plot(arima.sim(list(order=c(0,0,1), ma=.5), n=100), ylab="x",
main=(expression(MA(1)~~~theta==+.5)))
plot(arima.sim(list(order=c(0,0,1), ma=-.5), n=100), ylab="x",
main=(expression(MA(1)~~~theta==-.5)))
```

5.1.3 Autoregressive Moving Average Process

An ARMA process combines both autoregressive (AR) and moving average (MA) components. It is used for modeling time series data that have both autocorrelation and randomness.

```
par(mfrow = c(2,1))
plot(arima.sim(list(order=c(1,0,1), ar=.9,ma=.5), n=100), ylab="x",
main=(expression(ARMA(1,1)~~~phi==.9~~~theta==.5)))
plot(arima.sim(list(order=c(2,0,2), ar=c(.9,-.1),ma=c(.4,-.5)), n=100), ylab="x",
main=(expression(ARMA(2,2))))
```

5.1.4 ACF and PACF of the Model

Autocorrelation function (ACF) and partial autocorrelation function (PACF) are used to analyze the time series data and help identify the appropriate model order.

```
ACF = ARMAacf(ar=c(1.5,-.75), ma=0, 24)
PACF = ARMAacf(ar=c(1.5,-.75), ma=0, 24, pacf=TRUE)
par(mfrow=c(1,2))
plot(ACF, type="h", xlab="lag", ylim=c(-.8,1)); abline(h=0)
plot(PACF, type="h", xlab="lag", ylim=c(-.8,1)); abline(h=0)

ACF = ARMAacf(ar=0, ma=c(1.5,-.75), 24)
PACF = ARMAacf(ar=0, ma=c(1.5,-.75), 24, pacf=TRUE)
par(mfrow=c(1,2))
plot(ACF, type="h", xlab="lag", ylim=c(-.8,1)); abline(h=0)
plot(PACF, type="h", xlab="lag", ylim=c(-.8,1)); abline(h=0)
```

5.1.5 Fitting of Dow Jones Utilities Index

The Dow Jones Utilities Index is analyzed for its time series properties. The `acf2()` function is used for autocorrelation analysis.

```
library(itsmr)
library(astsa)
plot.ts(dowj,ylab='Utility Index',main='Dow Jones Utility Index')
acf2(dowj)
dowj_d1 <- diff(dowj)
plot.ts(dowj_d1,ylab='Utility Index',main='Dow Jones Utility Index')
acf2(dowj_d1)
```

5.1.6 Parameter Estimation Algorithms

Various parameter estimation techniques such as Yule-Walker, Burge, and Maximum Likelihood Estimation (MLE) are used to estimate the parameters of the model.

```
dowj_d1.ml = arma(dowj_d1,p=1,q=0);print(dowj_d1.ml)      # MLE (ARMA)
dowj_d1.at=autofit(dowj_d1,p=0:4,q=0:4);print(dowj_d1.at)  # Autofit by MLE
```

5.1.7 Fitting of Lake Huron (Raw)

Time series analysis of the Lake Huron data is performed using `acf2()` and ARMA models for parameter estimation.

```
plot.ts(LakeHuron)
acf2(LakeHuron)
huron.ml = arma(LakeHuron,p=1,q=1);print(huron.ml)      # MLE
huron.at=autofit(LakeHuron,p=0:5,q=0:5);print(huron.at) # Autofit by MLE
```

5.1.8 Checking the Residuals

The residuals of the fitted model are analyzed to ensure the adequacy of the model.

```
huron.aim = arima(LakeHuron, order=c(1,0,1))
plot(as.numeric(time(LakeHuron)),huron.aim$residuals, type="o",
xlab="Year", ylab="Residuals")
acf2(huron.aim$residuals)
```

5.1.9 Fitting Data with Trend

The Lake Huron data is detrended by subtracting the mean, and a linear regression is applied to fit the trend. The residuals are analyzed after removing the trend.

```
plot.ts(LakeHuron-570,type="o",ylab="level in feet")
trFit <- lm(LakeHuron-570~time(LakeHuron))
summary(trFit)
abline(trFit,col="red")
plot(as.numeric(time(LakeHuron)),trFit$residuals, type="o",
xlab="Year", ylab="Residuals")
```

5.1.10 Fitting Data After Removing Trend

The residuals from the trend removal are fitted using ARMA models, and the autocorrelations are analyzed.

```
plot.ts(trFit$residuals)
acf2(trFit$residuals)
huron.res.at=autofit(trFit$residuals,p=0:5,q=0:5);print(huron.res.at) # Autofit by MLE
huron.res.aim = arima(trFit$residuals, order=c(2,0,0))
plot(as.numeric(time(LakeHuron)),huron.res.aim$residuals,
type="o", xlab="Year", ylab="Residuals")
acf2(huron.res.aim$residuals)
```

6 06.TScode.R————————————————————————————-

Preparing the Environment

We begin by loading essential libraries:

```
library(survival)
library(MASS)
library(fitdistrplus)
```

Step 0: Data Loading and Exploration

The German Credit dataset is loaded and examined. Note that here, "Bad" customers are considered event occurrences.

```
myCredit = read.csv("GermanCredit.csv")
head(myCredit)
names(myCredit)
str(myCredit)
summary(myCredit)
```

Step 1: Preparing Right-Censored Data

Survival data requires two components:

- **Lifetime**: Time-to-event or censoring.
- **Status**: Event indicator (1 = event, 0 = censored).

```
Lifetime = myCredit$duration
Status = myCredit$credit_risk
CensorData = data.frame(cbind(Lifetime, Status))
SurvObject = Surv(myCredit$duration, (myCredit$credit_risk)==0)

left = Lifetime
right = Lifetime
right[which(Status == 0)] = NA
LRdata = data.frame(cbind(left, right))
```

Step 2: Parametric Survival Estimation (Without Covariates)

Here, we fit the survival function using MLE under various distributional assumptions:

- Gamma
- Weibull
- Log-normal
- Log-logistic

```
[language=R]
FitG = fitdistcens(LRdata, "gamma")
FitW = fitdistcens(LRdata, "weibull")
FitLN = fitdistcens(LRdata, "lnorm")
library(actuar)
FitLL = fitdistcens(LRdata, "llogis")
```

These fits are visualized as survival curves:

```
[language=R]
trange = 0:max(na.omit(LRdata[,2]))
plot(trange, 1 - pgamma(trange, shape = FitG$est[1],
rate = FitG$est[2]),type='l', col=2, ylim=c(0,1),
xlab="Time", ylab="Survival Probability")
lines(trange, 1 - pweibull(trange, shape = FitW$est[1],
scale = FitW$est[2]), col=4)
lines(trange, 1 - plnorm(trange, meanlog = FitLN$est[1],
sdlog = FitLN$est[2]), col=6)
lines(trange, 1 - pllogis(trange, shape = FitLL$est[1],
scale = FitLL$est[2]), col=8)
legend("bottomleft", c("Gamma", "Weibull", "Log-normal",
"Log-logistic"),col=c(2,4,6,8), lty=1, cex=0.8)
```

Step 3: Parametric Regression with Covariates (AFT Models)

The Accelerated Failure Time (AFT) model is a parametric approach to include covariates. We consider:

- Weibull Regression
- Log-normal Regression
- Log-logistic Regression

Weibull AFT Model

```
[language=R]
ResultW = survreg(SurvObject ~ age + amount + installment_rate+
number_credits + people_liable,data = myCredit, dist = "weibull")
summary(ResultW)
```

The shape parameter is the reciprocal of the scale parameter.

```
[language=R]
1 / summary(ResultW)$scale
```

Log-normal AFT Model

```
[language=R]
ResultLN = survreg(SurvObject ~ age + amount + installment_rate +
number_credits + people_liable,data = myCredit, dist = "lognormal")
summary(ResultLN)
```

Log-logistic AFT Model

```
[language=R]
ResultLL = survreg(SurvObject ~ age + amount + installment_rate+
number_credits + people_liable,data = myCredit, dist = "loglogistic")
summary(ResultLL)
```

Conclusion

This analysis demonstrates how different parametric survival models can be fitted using MLE, both with and without covariates. These techniques are useful for risk modeling and predicting time-to-event outcomes in financial and healthcare domains.